Neural Architecture Search: Lecture Notes

Sean Curow, Ryan Lee, Noah Worobec

April 23, 2026

1 Learning Objectives

The learning objectives of these lecture notes and the associated presentation are as follows:

- Identify the major stages and flow of NAS, explain them in the context of a search problem.
- Understand and Explain the consequences, reasoning, and motivations behind choosing a search space, search strategy, and evaluation strategy for NAS.
- Understand the potential computational cost of NAS
- Develop the conceptual framework for using NAS to solve a real-world problem

The target audience for this report is individuals with a beginner to intermediate background in the field of machine learning, who may be interested in pursuind the subjects of this report further.

2 Motivations and Pedagogical Approach

The topic of AutoML has become an extremely relevant and widely discussed one in recent years, especially with recent advancements and popularization of generative AI. Neural Architecture Search is an AutoML algorithm, and one that plays a significant role in the field. It has uses that range from natural language processing, to computer vision (image enhancement, classification, object detection), and many more. A key result of the algorithmic methods presented in this document is that they have shown great promise in finding and creating neural architectures that significantly outperform human-designed architectures of similar types [2]. While the computational cost of this algorithm is significant, so are the uses and benefits, making this one of the most critical topics in the field of AutoML.

The lecture notes in this document and the associated presentation present a pedagogical approach that aims to satisfy the learning objectives above through various methods. The most important aspect of this approach is the breakdown of the approach to designing an NAS algorithm, alongside the various design choices one can make and the consequences of these choices. By providing this breakdown and relating the structure to a known algorithmic structure, this approach aims to make a complicated and nuanced topic understandable to an individual with mid-level knowledge to the machine learning field. It also aims to enable this audience to pursue a minimal implementation of NAS on their own if it is their desire by breaking down different design choices that could be made when doing so. Finally, a brief computational cost analysis will make the audience aware of the resources necessary to pursue these methods on their own, and a presentation of our experimental results via visualizations and explanations, and finally an analysis of these results will round out the presentation by hopefully filling in remaining gaps in understanding.

This structure was chosen quite simply with the intent to provide as much breadth of knowledge on the topic as possible, so that our audience may obtain an understanding of background concepts and the basic structure of NAS. This will ideally provide any interested readers or listeners with a jumping off point to delve further into the topics of AutoML and NAS.

3 What is Neural Architecture Search (NAS)?

As discussed previously, NAS is part of the family of AutoML techniques, which is a family of techniques that automates the process of various machine learning algorithms and methods. This method, Neural Architecture Search, automates the creation, training, and selection of neural network architectures. Traditionally, the creation of neural architectures requires extensive research, design, and tweaking by human researchers and designers, which significantly delays the actual desired result of the process. This result is an "accurate" neural network that processes data in a desired format to produce a desired result that can either solve a problem or be evaluated and analyzed to further research and development of more, better neural networks. NAS solves this problem of wasted time, as it automates the tedious parts of the process, freeing up human researchers and designers to complete tasks that are more worth their time and energy.

The goal of NAS is to select and train an optimal neural network from a search space of neural architectures. The term optimal in this goal definition provides motivation for this problem as an optimization problem, which it most certainly is! One issue with human-designed networks is that in complex cases, it is quite difficult to create a truly optimal model. NAS solves this by running a search space of neural architectures through various algorithms that search for an optimal architecture in thar search space. There are various strategies and methods that can do this, and in the following sections they will be discussed in depth. For now though, the following expression is a common way to view NAS as an optimization problem:

$$\min_{a \in \mathcal{A}} \ \mathcal{L}_{\text{val}}(w^*(a), a), \ \text{ s.t. } \ w^*(a) = \text{argmin}_w \ \ \mathcal{L}_{\text{train}}(w, a)$$

Where $\mathcal{A}$ is the search space of neural architectures and a is a specific architecture, w represents the parameters of that architecture, and $\mathcal{L}_{\text{val}}$ and $\mathcal{L}_{\text{train}}$ are the validation and training loss respectively [1]. While this is the general approach, there are variants to it as well.

4 Why NAS?

Circling back a bit, the last section covers the main point of this section in brief: NAS handles the tedious tasks and frees up human resources for other, more pertinent tasks. This is the main attraction and main benefit, at least that is outwardly visual. However, it is important to mention that NAS is a very time-expensive algorithm, and will still take a very long time to run on its own. It does still free up human resources, but it hardly speeds up the process. However, there are other, very noteworthy aspects of neural architecture search that make it quite attractive.

The first of these is that neural architectures produced by NAS have been shown to outperform human designed neural architectures for the same use cases [2]. This is largely due to the optimization problem nature of NAS, and certian methods that lean heavily into this nature have proven to be the most reliable at outperforming human designs. A side effect of this is that those over-performing neural architectures are by necessity novel, which can lead to new insights and practices that went previously undiscovered.

The use cases for NAS are limited by the imagination and intuition of the user only, but there are some use cases that are quite commonly seen in implementation; namely natural language processing and computer vision, the latter being various methods of image processing, enhancement, classification, and more. In these tasks in particular, NAS has shown success in dealing with more complex data, and producing more complex and nuanced (and more accurate) solutions than humans have been able to deal with and produce in similar cases [2]. This speaks to the scalability of NAS, showing that it likely has far more uses than we have as of yet discovered for it.

5 NAS as a Search Problem

To assist in audience understanding of the NAS structure, let us compare it to a generic search problem. A search problem involves defining a set of possibilities called a search space, and then solving a computational problem to find a solution that solves a given goal. Think of a solving a maze: The search space involves all posssible locations in the maze as well as all legal movements of the player, the computational problem solves the goal of finding a path to the center (or exit) of the

maze. Treating it as an optimization problem would add the condition that the path is the shortest or quickest possible path.

To relate this to NAS, think of the set of all neural architectures as all possible paths in the maze (search space), and the computational problem is finding the path that produces the optimal result, which is the shortest path for the maze, and the most accurate neural architecture for NAS. The following sections break down the major steps of the NAS search problem and dive into each in detail. These steps are broken down as follows:

- The search space
- The search strategy
- The evaluation strategy

The generic algorithm for an NAS problem is as follows: Choose and define a search space, iterate through architectures in the search space by running the search strategy and evaluation strategy. Iterate until a chosen criterion is met, whether that be an "optimal" solution, iteration limit, or some accuracy limit [1]. Decisions made for each of these steps impacts the design and structure of the solution, and different design choices should be made for different problems. In particular, the search strategy and evaluation strategy have huge impact on computational cost and solution accuracy, and deserve special attention. The search space is also incredibly important, but can be derived relatively easily from the problem definition.

6 The Breakdown: Search Spaces

The search space is defined as the set of all neural architectures that the problem will root through in order to find a desired (perhaps optimal) solution. The search space is defined by a general neural architecture structure (usually define or at least limit the number of hidden layers, nodes per layer, etc), and the size of the space. Simpler problems will have a size in the hundreds to low millions, in terms of number of architectures. Complex big-data problems in real world applications, however, can have search space sizes exceeding 10^{20}. Necessarily, then, this space must be defined on a problem to problem basis. Defining a search space can be a tricky business, with numerous potential pitfalls to avoid. If little to no restrictions are imposed on a search space, it could be far too general and result in a wide variety and massive number of architectures, significantly slowing down search and evaluation and wasting time on architectures that would never have worked in the first place. If too many restrictions are imposed, human bias can be introduced, significantly influencing the final output, or reducing the size of the search space so much that it may as well be searched manually. Taking all of this into account, it is easy to see the massive implications that the search space has on the remainder of the NAS search problem.

7 The Breakdown: Search Strategies

This section answers the question: "How do we search the search space?" There are various strategies for doing so, and the most relevant and useful ones will be discussed in this section.

The first of these strategies is the simplest, yet sometimes most useful: Random search. This does exactly what its name suggests, selcting random architectures and evaluating them one by one to determine whether or not it meets the search problem's goal. It may seem like a rather "dumb" solution, wandering blindly through the search space until it happens upon a solution, but it has its uses: If the problem is an optimizable one, the entire search space is searchable, and the goal is the optimal solution, it is guaranteed to find the optimal solution in that search space (domain). This only works well for simple problems with small and non-complex search spaces, and efficiency of this strategy decreases quickly as size and complexity increase. This method is often used as a baseline for the other methods that will be discussed in this section.

The next strategy is one that has its merits, but that has suffered a fall from grace after newer, faster, and more accurate methods have emerged: Reinforcement learning. Reinforcement learning is a extremely costly practice on one model or network, so its easy to see how this can get out of hand very quickly with a large search space. Early endeavors resulted in computational costs of hundreds

Algorithm 1 General Reinforcement Learning NAS Algorithm

Input: Search space $\mathcal{A}$, number of iterations T.
Randomly initialize weights θ of the controller architecture.
for $t = 1, \ldots, T$ **do**
  Train architecture $a \sim \pi(a;\theta)$, randomly sampled from the controller policy $\pi(a;\theta)$.
  Update controller parameters θ by performing a gradient update $\nabla_\theta E_{a\sim\pi(a;\theta)}[\mathcal{L}_{\text{val}}(a)]$.
end for
Output: Architecture selected from the trained policy $\pi(a;\theta^*)$

Figure 1: The general reinforcement learning NAS algorithm

Algorithm 3 General Bayesian Optimization NAS Algorithm

Input: Search space $\mathcal{A}$, number of iterations T, acquisition function ϕ.
Randomly sample and train a population of architectures from the search space $\mathcal{A}$.
for $t = 1, \ldots, T$ **do**
  Train a surrogate model based on the current population.
  Select architecture a_t by maximizing $\phi(a)$, based on the surrogate model.
  Train architecture a_t and add it to the current population.
end for
Output: Architecture from the population with the highest validation accuracy.

Figure 2: The general bayesian optimization NAS algorithm

of GPU hours and hundreds of GPU's worth of memory. This strategy can succeed, although it is limited in its uses as it suffers with large search spaces. It works best with smaller search spaces that are complex in nature. The general algorithm can be seen in figure 1 [1].

Up next is the a more popular and largely successful [1] approach: Bayesian optimization. This method does a lot better computation cost-wise than reinforcement learning, and in practice, produces far better results for optimizable problems. It deals with training a probabilistic model to approximate the goal, and choosing models that maximize that model. This method works well with high-complexity architectures in terms of evaluation, as it simulates the goal as its evaluation step. The general algorithm can be seen in figure 2 [1].

Lastly, we have another popular and successful [1] approach: Evolutionary or genetic algorithms. These types of algorithms deal with a chosen "population" of architectures, sampling two at a time to produce a child, and then perhaps putting that child through a "mutation" to provide progress in the development of the architecture population. This method has been shown, on numerous occasions, to outperform random and reinforcement learning approaches on similar (or the same) problem types [1]. The general algorithm can be seen in figure 3 [1].

8 The Breakdown: Evaluation Strategies

One major consideration for NAS has to do with the performance evaluation part of the general algorithm To truly evaluate the performance of the chosen architecture it has to be trained, which is also handled in this step. Training and evaluating a complex neural architecture is already extremely computationally expensive, but in the case of NAS one has to search through at the minimum hundereds and at the maximum billions of neural architectures, making the problem exponentially worse. Fortunately, through research and experimentation, various methods have been developed to mitigate this issue, and the most relevant ones will be briefly discussed in this section.

- The first method to discuss is the use of proxy tasks. This method involves coming up with a simpler, but similar proxy task to the main problem, and training/evaluating the models on that. Depending on the problem, one could cut training and evaluation time down significantly

Algorithm 2 General Evolutionary NAS Algorithm

Input: Search space $\mathcal{A}$, number of iterations T.
Randomly sample and train a population of architectures from the search space $\mathcal{A}$.
for $t = 1, \ldots, T$ **do**
  Sample (based on accuracy) a set of parent architectures from the population.
  Mutate the parent architectures to generate children architectures, and train them.
  Add the children to the population, and kill off the architectures that are the oldest (or have the lowest accuracy) among the current population.
end for
Output: Architecture from the population with the highest validation accuracy.

Figure 3: The general evolutionary or genetic NAS algorithm

compared to training on the full task. This is best suited, obviously, for problems that are intuitively able to be simplified.

- The next method to discuss is the general idea of scaling down as it pertains to various aspects of the architectures and problem. Limiting aspects such as the size of the training dataset, number of allowed iterations for iterative training methods, or even scaling down the number of parameters in a model can all lead to significantly reduced cost and time to train and evaluate architectures.
- This method is often referred to as weight inheritance. This method is a very popular one in the research areas of this field, and involves models inheriting weights from fully or partially trained previous models. This can work by putting each architecture far closer to its goal at the start of its training cycle, again greatly reducing cost and time.
- Finally, another method involves extrapolating the learning curve of the chosen architecture as it trains. If a model is predicted to be below a certain benchmark (say, the best recorded performance thus far) by a certain point in its training, then cut the training and discard the architecture, moving on to the next.

All of the above evaluation strategies for neural architectures, like search strategies, have varying degrees of usefulness for different types of problems and architectures, and it is vital to choose which to use carefully. All options have their inherent risks, and mitigating these is crucial to your final result.

9 Computational Cost

By nature, AutoML techniques are typically extremely computationally expensive, as they take already costly algorithms and then automate their design and creation, potentially millions of times over a single run of the technique. Neural Architecture Search is no exception to this rule. Traditional methods can devour hundreds of GPU days and hundreds of GPU's worth of memory, or potentially more based on the complexity of the problem at hand. Even modern day methods such as one-shot that can reduce these astronomical cost values drastically still consume days of time and multiple GPU's of memory to run on simple setups. One shot is a method of NAS that greatly simplifies the process by creating and training a "super-network" of neural networks from the search space, instead of building individual architectures one by one. The individual architectures from the search space are then evaluated by performing evaluations on their corresponding sub-network inside the super-network. However, as revolutionary and efficient as one shot may be, NAS remains one of the most computationally expensive AutoML methods out there.

10 Experimental Results and Analysis

To best motivate NAS as a method, and to help our audience understand the potential benefits and drawbacks, we have conducted experiments around the subject. Our experiments revolved around using a human-designed baseline model to evaluate the relative performance of a minimal NAS implementation that revolves around a random search strategy. The aim is to show its potential benefits and drawbacks when compared to human design. To achieve this, the NAS algorithm was limited to strictly keep its models under the human designed parameter budget, which ended up around 100K parameters. The defined search space was architectures selected via a list of randomly chosen layer operations (3x3 convolution, 5x5 convolution, 2x2 pooling) that were then compiled into a valid PyTorch model. The evaluation strategy was then to use a proxy function with a lower number of epochs to minimize time spent evaluating models. More on the technical side of things, There were 120 models in the search space that were evaluated via a 2 epoch proxy task. Once the "winning" NAS model was found, it was trained alongside the baseline model for 25 epochs, using the optimizer Adam with a learning rate of 0.001. The CIFAR-10 dataset was the one selected for this task. Finally, to measure loss, cross entropy was the selected loss function.

The winning NAS model ended up with just over 38K parameters, and the layers were a 3x3 convolution, pooling, 3x3 convolution, pooling, pooling, and finally, a last 3x3 convolution.

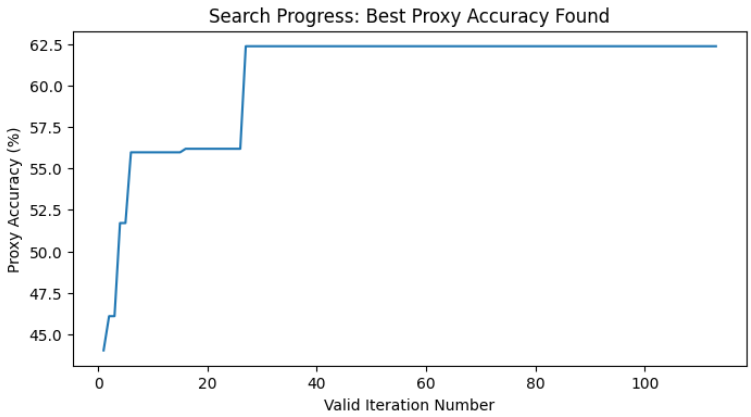

Figure 4: Graph showing proxy accuracy over iterations

In figure 4 it can be seen that the proxy task accuracy made large jumps in accuracy early on in the iterations before settling on a high accuracy around iteration 30.

In figure 5 we see a plot of all the models comparing their parameter count to their accuracy. The human baseline model has the highest number of parameters, but it can be seen that many of the NAS candidates managed to beat its accuracy despite the disadvantage in parameter count. The winning NAS model is seen with less than half the number of parameters than the human baseline, but a fairly significantly improved accuracy.

In figure 6, we see a comparison of the efficiency of the winning NAS model vs the human designed baseline model. While their actual accuracy may not be too different, the NAS winner was far more efficient, using less than half of the parameters. This is extremely evident, looking at this graph.

Finally, in figure 7 we see a comparison of the human baseline and NAS winner, specifically comparing their test accuracies and relative model size. While their test accuracies are quite comparable, with the human design winning by a small amount, it is clear to see that this was at the cost of a vastly larger model size.

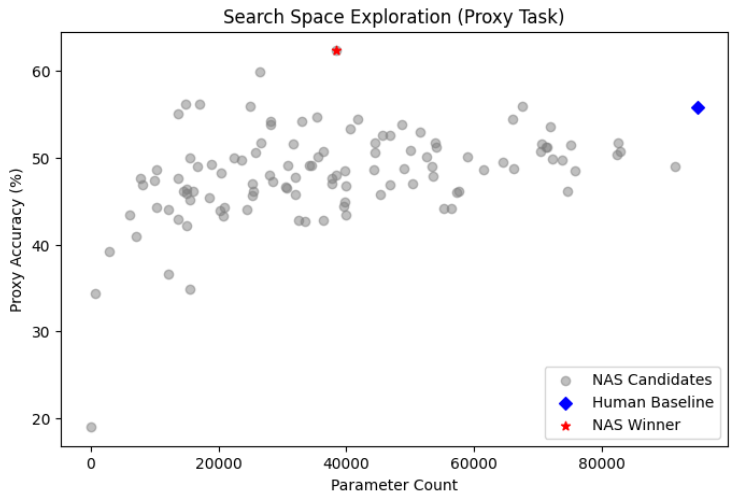

Figure 5: Graph showing the accuracy of various models vs the number of parameters

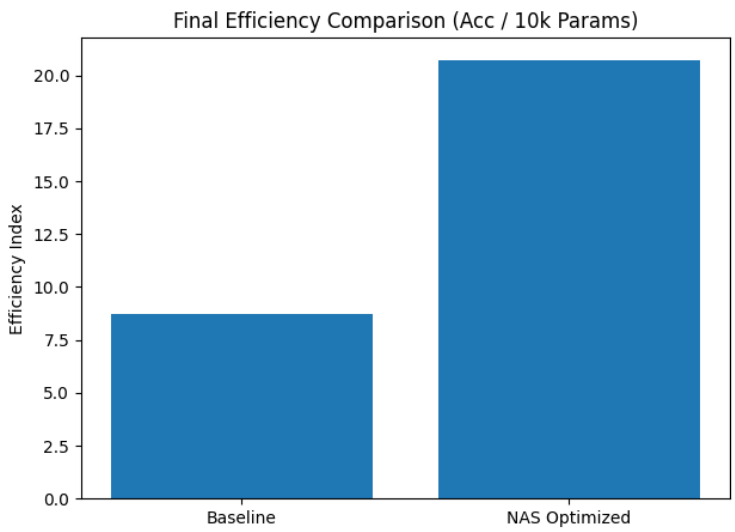

Figure 6: Graph showing the accuracy normalized by number of params

11 Limitations and Potential Expansions

In our experiments, it is important to note that we were limited to the small search space and rudimentary search and evaluation strategies by the resources available to us. Running anything significantly larger or more complicated on a standard modern laptop would take much more time than was reasonable for the scope of this project, and thus we were limited to the experiments shown in the previous section. One-GPU machines are limited to minimal implementations of NAS, such as our own. However, a more sophisticated setup with even a small GPU bank is capable of much, much more. Real world problems could be solved with real world datasets given more resources. Different methods for searches and evaluation could be used, larger more complex datasets and search spaces are possible, along with much more. At the end of the day, we hope that this document has provided our audience with a good jumping off point for the subject, but it is vital to know that in the area of NAs there is still much to learn!

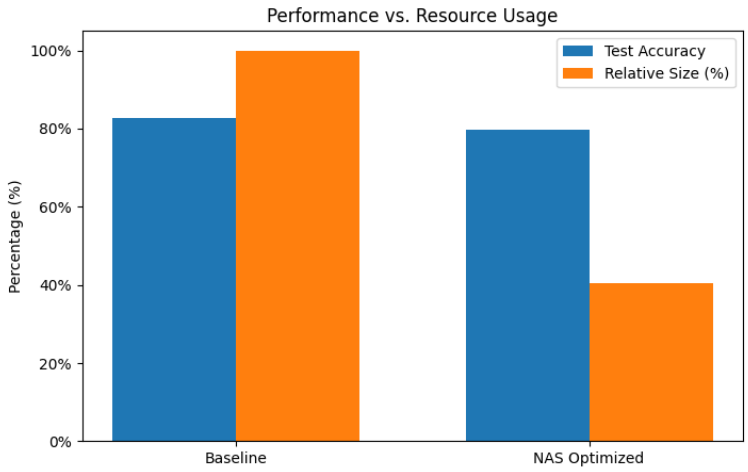

Figure 7: Graph showing test accuracy vs relative model size

References

[1] Colin White, M.S., et al.: Neural architecture search: Insights from 1000 papers. Tech. rep., Abacus.AI, University of Freiburg, Sailyond Technology & Research Institute of Tsinghua University, Bosch Center for Artificial Intelligence, Microsoft Research (2022), https://arxiv.org/pdf/2301.08727, last accessed Mar 24, 2026

[2] Sharma, I.K.: Neural architecture search algorithm (2025), https://www.geeksforgeeks.org/deep-learning/neural-architecture-and-search-methods/, last accessed Mar 24, 2026